

RNS-CKKS Implementation in Rust

Mathematical Foundations and Algorithms

CKKS Implementation Documentation

March 2026

Contents

1 Introduction

This document provides a detailed mathematical description of the algorithms implemented in our Rust-based RNS-CKKS homomorphic encryption library. The CKKS scheme (Cheon-Kim-Kim-Song) enables approximate arithmetic on encrypted data, making it particularly suitable for machine learning and statistical computations on encrypted data.

Our implementation follows the RNS (Residue Number System) variant, which improves computational efficiency by representing large integers as tuples of residues modulo smaller coprime integers.

2 Modular Arithmetic

All operations in CKKS are performed modulo prime numbers. This section describes the fundamental modular arithmetic operations.

2.1 Modular Addition and Subtraction

Definition 1 (Modular Addition). *For integers a, b and modulus $m > 0$:*

$$(a + b) \bmod m = r \quad \text{where } r \in [0, m) \text{ and } m \mid (a + b - r)$$

Algorithm 1 Modular Addition

```
1: function MODADD( $a, b, m$ )  
2:   return  $(a + b) \bmod m$  ▷ Use 128-bit intermediate to prevent overflow  
3: end function
```

Definition 2 (Modular Subtraction). *For integers a, b and modulus $m > 0$:*

$$(a - b) \bmod m = \begin{cases} (a - b) \bmod m & \text{if } a \geq b \\ m - ((b - a) \bmod m) & \text{if } a < b \end{cases}$$

Algorithm 2 Modular Subtraction

```
1: function MODSUB( $a, b, m$ )  
2:   if  $a \geq b$  then  
3:     return  $(a - b) \bmod m$   
4:   else  
5:     return  $m - ((b - a) \bmod m)$   
6:   end if  
7: end function
```

Complexity: $O(1)$ for both operations.

2.2 Modular Multiplication

Definition 3 (Modular Multiplication). *For integers a, b and modulus $m > 0$:*

$$(a \cdot b) \bmod m = r \quad \text{where } r \in [0, m) \text{ and } m \mid (ab - r)$$

Algorithm 3 Modular Multiplication

```

1: function MODMUL( $a, b, m$ )
2:   return  $(a \times b) \bmod m$            ▷ Use 128-bit intermediate:  $a, b < 2^{64}$ , so  $ab < 2^{128}$ 
3: end function

```

Complexity: $O(1)$ (constant time for fixed-width integers).

2.3 Modular Exponentiation (Square-and-Multiply)

Naive exponentiation a^n requires $n - 1$ multiplications, which is exponential in the bit-length of n . The **binary exponentiation** (square-and-multiply) algorithm computes $a^n \bmod m$ in $O(\log n)$ multiplications.

Theorem 1. For any integer n with binary representation $n = \sum_{i=0}^k b_i 2^i$ where $b_i \in \{0, 1\}$:

$$a^n = a^{\sum_{i=0}^k b_i 2^i} = \prod_{i: b_i=1} a^{2^i}$$

Algorithm 4 Binary Exponentiation (Square-and-Multiply)

```

1: function MODEXP(base, exp,  $m$ )
2:   if  $m = 1$  then
3:     return 0
4:   end if
5:   result  $\leftarrow$  1
6:   base  $\leftarrow$  base mod  $m$ 
7:   while exp  $> 0$  do
8:     if exp  $\wedge 1 = 1$  then           ▷ If lowest bit is 1
9:       result  $\leftarrow$  (result  $\times$  base) mod  $m$ 
10:    end if
11:    base  $\leftarrow$  (base  $\times$  base) mod  $m$            ▷ Square
12:    exp  $\leftarrow$  exp  $\gg$  1           ▷ Right shift (divide by 2)
13:  end while
14:  return result
15: end function

```

Example: Computing $2^{13} \bmod 1000$:

- $13 = (1101)_2 = 8 + 4 + 1$
- $2^{13} = 2^8 \cdot 2^4 \cdot 2^1 = 256 \cdot 16 \cdot 2 = 8192$
- $8192 \bmod 1000 = 192$

Complexity: $O(\log n)$ multiplications, where n is the exponent.

2.4 Modular Inverse

Definition 4 (Modular Inverse). The modular inverse of a modulo m is an integer x such that:

$$a \cdot x \equiv 1 \pmod{m}$$

The inverse exists if and only if $\gcd(a, m) = 1$.

2.4.1 Method 1: Fermat's Little Theorem (Prime Modulus)

Theorem 2 (Fermat's Little Theorem). *For prime p and integer a not divisible by p :*

$$a^{p-1} \equiv 1 \pmod{p}$$

Therefore:

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

Algorithm 5 Modular Inverse via Fermat's Little Theorem

```

1: function MODINV( $a, p$ )                                     ▷  $p$  must be prime
2:   return MODEXP( $a, p - 2, p$ )
3: end function

```

Complexity: $O(\log p)$ using binary exponentiation.

2.4.2 Method 2: Extended Euclidean Algorithm (General Case)

Theorem 3 (Bézout's Identity). *For any integers a, b , there exist integers x, y such that:*

$$ax + by = \gcd(a, b)$$

If $\gcd(a, m) = 1$, then $ax + my = 1$, which means $ax \equiv 1 \pmod{m}$.

Algorithm 6 Extended Euclidean Algorithm

```

1: function EXTENDEDGCD( $a, b$ )
2:   if  $a = 0$  then
3:     return ( $b, 0, 1$ )                                     ▷ Returns ( $\gcd, x, y$ )
4:   end if
5:   ( $\gcd, x_1, y_1$ )  $\leftarrow$  EXTENDEDGCD( $b \bmod a, a$ )
6:    $x \leftarrow y_1 - \lfloor b/a \rfloor \cdot x_1$ 
7:    $y \leftarrow x_1$ 
8:   return ( $\gcd, x, y$ )
9: end function

```

Complexity: $O(\log(\min(a, b)))$.

2.5 Miller-Rabin Primality Test

The Miller-Rabin test is a probabilistic primality test based on the following:

Theorem 4. *Let $n > 2$ be an odd integer. Write $n - 1 = 2^r \cdot d$ where d is odd. For any a coprime to n , if n is prime, then either:*

1. $a^d \equiv 1 \pmod{n}$, or
2. $a^{2^i \cdot d} \equiv -1 \pmod{n}$ for some $0 \leq i < r$

Algorithm 7 Miller-Rabin Primality Test

```
1: function ISPRIME( $n$ )
2:   if  $n < 2$  then return false
3:   end if
4:   if  $n = 2$  or  $n = 3$  then return true
5:   end if
6:   if  $n \bmod 2 = 0$  then return false
7:   end if
8:   Write  $n - 1 = 2^r \cdot d$  with  $d$  odd
9:   witnesses  $\leftarrow [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]$ 
10:  for each  $a$  in witnesses do
11:    if  $a \geq n$  then continue
12:    end if
13:     $x \leftarrow a^d \bmod n$ 
14:    if  $x = 1$  or  $x = n - 1$  then continue
15:    end if
16:    composite  $\leftarrow$  true
17:    for  $i \leftarrow 0$  to  $r - 2$  do
18:       $x \leftarrow x^2 \bmod n$ 
19:      if  $x = n - 1$  then
20:        composite  $\leftarrow$  false
21:        break
22:      end if
23:    end for
24:    if composite then return false
25:    end if
26:  end for
27:  return true
28: end function
```

Remark 1. With the witnesses $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$, the test is deterministic for all $n < 2^{64}$.

Complexity: $O(k \log^2 n)$ where k is the number of witnesses.

2.6 Finding Primitive Roots of Unity

Definition 5 (Primitive n -th Root of Unity). An element $\omega \in \mathbb{Z}_p$ is a primitive n -th root of unity if:

1. $\omega^n \equiv 1 \pmod{p}$
2. $\omega^k \not\equiv 1 \pmod{p}$ for all $0 < k < n$

Theorem 5. A primitive n -th root of unity exists in $\mathbb{Z}_p$ if and only if $n \mid (p - 1)$.

Algorithm 8 Find Primitive Root of Unity

```
1: function FINDPRIMITIVEROOT( $n, p$ )
2:   if  $(p - 1) \bmod n \neq 0$  then
3:     return None ▷  $n$  must divide  $p - 1$ 
4:   end if
5:    $g \leftarrow \text{FINDGENERATOR}(p)$  ▷ Primitive root of  $\mathbb{Z}_p^*$ 
6:   return  $g^{(p-1)/n} \bmod p$ 
7: end function
```

Algorithm 9 Find Generator of $\mathbb{Z}_p^*$

```
1: function FINDGENERATOR( $p$ )
2:   factors  $\leftarrow$  prime factors of  $(p - 1)$ 
3:   for  $g \leftarrow 2$  to  $p - 1$  do
4:     isGenerator  $\leftarrow$  true
5:     for each prime factor  $q$  of  $(p - 1)$  do
6:       if  $g^{(p-1)/q} \equiv 1 \pmod{p}$  then
7:         isGenerator  $\leftarrow$  false
8:         break
9:       end if
10:    end for
11:    if isGenerator then
12:      return  $g$ 
13:    end if
14:  end for
15: end function
```

3 Number Theoretic Transform (NTT)

The Number Theoretic Transform is the modular arithmetic analogue of the Fast Fourier Transform (FFT). It enables polynomial multiplication in $O(n \log n)$ time instead of $O(n^2)$.

3.1 Mathematical Foundation

Definition 6 (NTT). *Let ω be a primitive n -th root of unity in $\mathbb{Z}_p$. The NTT of a vector $(a_0, a_1, \dots, a_{n-1})$ is:*

$$A_k = \sum_{j=0}^{n-1} a_j \omega^{jk} \pmod{p}, \quad k = 0, 1, \dots, n-1$$

Definition 7 (Inverse NTT). *The inverse NTT is:*

$$a_j = \frac{1}{n} \sum_{k=0}^{n-1} A_k \omega^{-jk} \pmod{p}, \quad j = 0, 1, \dots, n-1$$

Theorem 6 (Convolution Theorem). *For polynomials $a(x)$ and $b(x)$, if $A = \text{NTT}(a)$ and $B = \text{NTT}(b)$, then:*

$$\text{NTT}^{-1}(A \odot B) = a \cdot b$$

where $\odot$ denotes component-wise multiplication.

3.2 NTT Context Pre-computation

To avoid redundant computation, we pre-compute and store:

- ω : primitive $2n$ -th root of unity (for negacyclic convolution)
- ω^{-1} : inverse of ω
- n^{-1} : inverse of n (for scaling in inverse NTT)
- $[\omega^0, \omega^1, \omega^2, \dots, \omega^{n-1}]$: pre-computed powers
- $[\omega^{-0}, \omega^{-1}, \omega^{-2}, \dots, \omega^{-(n-1)}]$: inverse powers

3.3 Bit-Reversal Permutation

The Cooley-Tukey algorithm requires reordering the input by bit-reversal.

Definition 8 (Bit-Reversal). *For an index i with $\log_2 n$ bits, the bit-reversed index $\text{rev}(i)$ is obtained by reversing the binary representation of i .*

Example for $n = 8$:

Index	Binary	Bit-Reversed
0	000	000 $\rightarrow$ 0
1	001	100 $\rightarrow$ 4
2	010	010 $\rightarrow$ 2
3	011	110 $\rightarrow$ 6
4	100	001 $\rightarrow$ 1
5	101	101 $\rightarrow$ 5
6	110	011 $\rightarrow$ 3
7	111	111 $\rightarrow$ 7

Algorithm 10 Bit-Reversal

```

1: function BITREVERSE( $x, \log_n$ )
2:   result  $\leftarrow$  0
3:   for  $i \leftarrow 0$  to  $\log_n - 1$  do
4:     result  $\leftarrow$  (result  $\ll$  1) | ( $x$  & 1)
5:      $x \leftarrow x \gg 1$ 
6:   end for
7:   return result
8: end function

```

3.4 Cooley-Tukey Forward NTT

The Cooley-Tukey algorithm is a divide-and-conquer approach that recursively splits the DFT into smaller DFTs.

Algorithm 11 Cooley-Tukey Iterative NTT

```
1: function NTTFORWARD( $a, \omega, p$ )
2:    $n \leftarrow |a|$ 
3:    $\log_n \leftarrow \log_2 n$ 
4:    $A \leftarrow \text{BITREVERSECOPY}(a, \log_n)$ 
5:    $m \leftarrow 1$ 
6:   for  $s \leftarrow 0$  to  $\log_n - 1$  do
7:      $m_2 \leftarrow 2m$ 
8:      $\text{step} \leftarrow n/m_2$ 
9:     for  $k \leftarrow 0$  to  $n - 1$  step  $m_2$  do
10:       $w \leftarrow 1$ 
11:      for  $j \leftarrow 0$  to  $m - 1$  do
12:         $t \leftarrow w \cdot A[k + j + m] \bmod p$  ▷ Butterfly
13:         $u \leftarrow A[k + j]$ 
14:         $A[k + j] \leftarrow (u + t) \bmod p$ 
15:         $A[k + j + m] \leftarrow (u - t) \bmod p$ 
16:         $w \leftarrow w \cdot \omega^{\text{step}} \bmod p$  ▷ Update twiddle
17:      end for
18:    end for
19:     $m \leftarrow m_2$ 
20:  end for
21:  return  $A$ 
22: end function
```

Butterfly Operation: The core operation combines two elements using:

$$A[k + j] \leftarrow A[k + j] + \omega^j \cdot A[k + j + m] \quad (1)$$

$$A[k + j + m] \leftarrow A[k + j] - \omega^j \cdot A[k + j + m] \quad (2)$$

Complexity: $O(n \log n)$ multiplications and additions.

3.5 Gentleman-Sande Inverse NTT

The inverse NTT uses ω^{-1} instead of ω and scales by n^{-1} at the end.

Algorithm 12 Inverse NTT

```
1: function NTTINVERSE( $A, \omega^{-1}, n^{-1}, p$ )
2:    $a \leftarrow \text{NTTFORWARD}(A, \omega^{-1}, p)$  ▷ Same structure with  $\omega^{-1}$ 
3:   for  $i \leftarrow 0$  to  $n - 1$  do
4:      $a[i] \leftarrow a[i] \cdot n^{-1} \bmod p$  ▷ Scale by  $1/n$ 
5:   end for
6:   return  $a$ 
7: end function
```

3.6 Polynomial Multiplication via NTT

To multiply polynomials $a(x)$ and $b(x)$:

Algorithm 13 NTT Polynomial Multiplication

```
1: function NTTMULTIPLY( $a, b$ )
2:    $A \leftarrow \text{NTTFORWARD}(a)$ 
3:    $B \leftarrow \text{NTTFORWARD}(b)$ 
4:    $C[i] \leftarrow A[i] \cdot B[i] \bmod p$  for all  $i$  ▷ Point-wise multiply
5:   return NTTINVERSE( $C$ )
6: end function
```

Total Complexity: $O(n \log n)$ vs. $O(n^2)$ for naive multiplication.

4 Polynomial Ring Operations

In CKKS, we work in the polynomial ring:

$$R_q = \mathbb{Z}_q[X]/(X^N + 1)$$

where N is a power of 2 and q is the ciphertext modulus.

4.1 The Ring Structure

Definition 9 (Polynomial Ring R_q). *Elements are polynomials of degree $< N$ with coefficients in $\mathbb{Z}_q$:*

$$a(X) = \sum_{i=0}^{N-1} a_i X^i, \quad a_i \in \mathbb{Z}_q$$

The quotient by $(X^N + 1)$ imposes:

$$X^N \equiv -1 \pmod{X^N + 1}$$

4.2 Polynomial Addition and Subtraction

Addition and subtraction are coefficient-wise:

$$(a + b)_i = a_i + b_i \bmod q$$

$$(a - b)_i = a_i - b_i \bmod q$$

Complexity: $O(N)$.

4.3 Scalar Multiplication

$$(c \cdot a)_i = c \cdot a_i \bmod q$$

Complexity: $O(N)$.

4.4 Polynomial Negation

$$(-a)_i = \begin{cases} 0 & \text{if } a_i = 0 \\ q - a_i & \text{otherwise} \end{cases}$$

4.5 Naive Polynomial Multiplication (Negacyclic)

For the ring $\mathbb{Z}_q[X]/(X^N + 1)$, multiplication wraps with sign change:

Algorithm 14 Naive Polynomial Multiplication in R_q

```

1: function NAIVEMUL( $a, b, q, N$ )
2:    $c \leftarrow [0, 0, \dots, 0]$   $\triangleright N$  zeros
3:   for  $i \leftarrow 0$  to  $N - 1$  do
4:     for  $j \leftarrow 0$  to  $N - 1$  do
5:        $\text{idx} \leftarrow i + j$ 
6:        $\text{product} \leftarrow a_i \cdot b_j \bmod q$ 
7:       if  $\text{idx} < N$  then
8:          $c[\text{idx}] \leftarrow (c[\text{idx}] + \text{product}) \bmod q$ 
9:       else
10:         $c[\text{idx} - N] \leftarrow (c[\text{idx} - N] - \text{product}) \bmod q$   $\triangleright X^N = -1$ 
11:      end if
12:    end for
13:  end for
14:  return  $c$ 
15: end function

```

Example: In $\mathbb{Z}_{17}[X]/(X^4 + 1)$:

$$X^3 \cdot X = X^4 = -1 \equiv 16 \pmod{17}$$

Complexity: $O(N^2)$ — use NTT for better performance.

5 Residue Number System (RNS)

RNS enables efficient computation on large integers by representing them via their residues modulo several smaller coprime moduli.

5.1 Mathematical Foundation

Theorem 7 (Chinese Remainder Theorem). *Let $q_0, q_1, \dots, q_L$ be pairwise coprime integers and $Q = \prod_{i=0}^L q_i$. There is a ring isomorphism:*

$$\mathbb{Z}_Q \cong \mathbb{Z}_{q_0} \times \mathbb{Z}_{q_1} \times \dots \times \mathbb{Z}_{q_L}$$

Any $x \in \mathbb{Z}_Q$ is uniquely determined by $(x \bmod q_0, x \bmod q_1, \dots, x \bmod q_L)$.

5.2 RNS Polynomial Representation

An RNS polynomial stores a polynomial's residues for each prime in the modulus chain:

$$\text{RNS}(a) = (a \bmod q_0, a \bmod q_1, \dots, a \bmod q_L)$$

Advantages:

- Arithmetic on large numbers reduces to parallel arithmetic on small numbers
- Each component fits in a machine word (64-bit)
- Operations can be parallelized across components

5.3 RNS Arithmetic

Addition, subtraction, and multiplication in RNS are component-wise:

$$\text{RNS}(a + b) = (\text{RNS}(a) + \text{RNS}(b)) \bmod \vec{q} \quad (3)$$

$$\text{RNS}(a - b) = (\text{RNS}(a) - \text{RNS}(b)) \bmod \vec{q} \quad (4)$$

$$\text{RNS}(a \cdot b) = (\text{RNS}(a) \cdot \text{RNS}(b)) \bmod \vec{q} \quad (5)$$

6 NTT-Friendly Prime Generation

For NTT to work efficiently, we need primes p satisfying:

$$2N \mid (p - 1)$$

This ensures primitive $2N$ -th roots of unity exist in $\mathbb{Z}_p$.

Algorithm 15 Generate NTT-Friendly Prime Chain

```

1: function GENERATEPRIMECHAIN( $N$ , count, bits)
2:   primes  $\leftarrow []$ 
3:    $k \leftarrow (2^{\text{bits}} - 1)/(2N)$  ▷ Start from largest candidate
4:   while |primes| < count and  $k > 0$  do
5:     candidate  $\leftarrow k \cdot 2N + 1$ 
6:     if ISPRIME(candidate) then
7:       primes.append(candidate)
8:     end if
9:      $k \leftarrow k - 1$ 
10:  end while
11:  return primes
12: end function

```

Form of NTT-friendly primes: $p = k \cdot 2N + 1$ for some integer k .

Examples:

- For $N = 16$: primes of form $32k + 1$
- $p = 97 = 3 \cdot 32 + 1$
- $p = 193 = 6 \cdot 32 + 1$

7 Complexity Summary

Operation	Time Complexity	Space Complexity
Modular Add/Sub/Mul	$O(1)$	$O(1)$
Modular Exponentiation	$O(\log n)$	$O(1)$
Modular Inverse (Fermat)	$O(\log p)$	$O(1)$
Extended GCD	$O(\log \min(a, b))$	$O(\log \min(a, b))$
Miller-Rabin Test	$O(k \log^2 n)$	$O(1)$
Bit Reversal	$O(n)$	$O(n)$
Forward NTT	$O(n \log n)$	$O(n)$
Inverse NTT	$O(n \log n)$	$O(n)$
NTT Multiply	$O(n \log n)$	$O(n)$
Naive Poly Multiply	$O(n^2)$	$O(n)$
Polynomial Add/Sub	$O(n)$	$O(n)$

8 Correctness Verification

The implementation includes unit tests verifying:

1. **Modular arithmetic:** Overflow handling, edge cases
2. **Modular inverse:** $a \cdot a^{-1} \equiv 1 \pmod{p}$
3. **Primality:** Known primes and composites
4. **Primitive roots:** $\omega^n \equiv 1, \omega^{n/2} \not\equiv 1$
5. **NTT round-trip:** $\text{INTT}(\text{NTT}(a)) = a$
6. **Polynomial operations:** Arithmetic identities
7. **Negacyclic convolution:** $X^N \equiv -1$

9 References

1. Cheon, J. H., Kim, A., Kim, M., & Song, Y. (2017). Homomorphic encryption for arithmetic of approximate numbers. *ASIACRYPT 2017*.
2. Cooley, J. W., & Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*.
3. Garner, H. L. (1959). The residue number system. *IRE Transactions on Electronic Computers*.
4. Miller, G. L. (1976). Riemann's hypothesis and tests for primality. *Journal of Computer and System Sciences*.
5. The Rust Programming Language. <https://doc.rust-lang.org/book/>